Department of CSE (Artificial Intelligence)

LAB MANUAL

Machine Learning

(23UAIPCP2404)

(2023 PATTERN)

Class: S. Y. Btech
Term:II

Assignment No. 1

Title: Prediction of Employee Salary Based on Years of Experience Using Simple Linear Regression

Aim: A company wants to analyze the relationship between an employee's years of experience and their salary. Develop a simple linear regression model that predicts the salary of an employee based on the number of years they have worked. Additionally, evaluate the performance of the model using standard regression metrics.

Software/libraries required:

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Outcomes of Assignment:

1. Understand the concept of Simple Linear Regression.
2. Develop a regression model to predict salary based on experience.
3. Interpret regression coefficients (slope and intercept).
4. Evaluate model performance using regression metrics.
5. Visualize regression results using graphical representation.

Theory:

1. Introduction to Simple Linear Regression

Simple Linear Regression is a supervised machine learning algorithm used to model the relationship between:

- **Independent Variable (X)** → Years of Experience
- **Dependent Variable (Y)** → Salary

It assumes a linear relationship between X and Y.

2. Mathematical Model

The linear regression equation is:

$$Y = \beta_0 + \beta_1 X$$

Where:

- Y= Predicted Salary
- X= Years of Experience
- β_0= Intercept (Salary when experience = 0)
- β_1= Slope (Increase in salary per additional year of experience)

3. How the Model Works
The model finds the best-fit line that minimizes the error between actual and predicted values.
Error Calculation
Error = Actual Salary − Predicted Salary
To minimize error, we use:
Cost Function (Mean Squared Error)

$$MSE = \frac{1}{n}\sum(Y_i - \hat{Y}_i)^2$$

The algorithm minimizes this MSE to find optimal values of $\beta_0$and β_1.

4. Diagram Representation
Scatter Plot with Regression Line

```
Salary |
     |                *
     |            *
     |        *
     |    *
     |  *
     |_____________________________
        Years of Experience
```

Figure 1: Scatter Plot with Regression Line

- Dots (*) represent actual employee data.
- Straight line represents predicted salary trend.
- The line shows positive linear relationship.

5. Assumptions of Linear Regression
1. Linearity
2. Independence of errors
3. Homoscedasticity (constant variance)
4. Normal distribution of errors

6. Model Evaluation Metrics
To evaluate performance, the following metrics are used:

a. **Mean Absolute Error (MAE)**

$$MAE = \frac{1}{n}\sum | Y_i - \hat{Y}_i |$$

- Measures average absolute error.
- Lower value = Better model.

b. **Mean Squared Error (MSE)**

$$MSE = \frac{1}{n}\sum(Y_i - \hat{Y}_i)^2$$

- Penalizes large errors.

- Lower value = Better performance.

c. Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{MSE}$$

- Interpretable in same units as salary.

d. R-Squared (Coefficient of Determination)

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

- Measures how well the model explains variance.
- Value ranges from 0 to 1.
- Closer to 1 = Better fit.

7. Steps to Build the Model

1. Import libraries
2. Load dataset
3. Split data into training and testing sets
4. Train Linear Regression model
5. Predict salary
6. Evaluate performance
7. Visualize regression line

Sample Algorithm

1. Start
2. Load dataset (Experience, Salary)
3. Split into training & testing data
4. Apply Linear Regression
5. Fit model
6. Predict salary
7. Calculate MAE, MSE, RMSE, R^2
8. Display results
9. Stop

Oral Questions:

1. What is Simple Linear Regression?
2. What is the difference between dependent and independent variables?
3. What does the slope represent in this model?
4. What is the role of the intercept?
5. What is Mean Squared Error?
6. Why is RMSE preferred over MSE?
7. What does R^2 indicate?
8. What are the assumptions of linear regression?
9. What happens if the relationship is not linear?

10. How can you improve a regression model?

Conclusion:

_____
_____
_____
_____

Assignment No. 2

Aim: Study and Implement Multiple Linear Regression.

Software/libraries required:
- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn

Outcomes of Assignment:
1. Understand the concept of Multiple Linear Regression (MLR).
2. Identify dependent and multiple independent variables in a dataset.
3. Build and train a Multiple Linear Regression model.
4. Interpret regression coefficients and model significance.
5. Evaluate model performance using standard regression metrics.

Theory:

1. Introduction to Multiple Linear Regression

Multiple Linear Regression is an extension of Simple Linear Regression. It is used when the dependent variable depends on more than one independent variable.

Example:

Predicting House Price based on:
- Area (sq.ft)
- Number of Bedrooms
- Location Score
- Age of Property

Here,
- Dependent Variable (Y) → House Price
- Independent Variables (X_1, X_2, X_3, …)

2. Mathematical Model

The general equation of Multiple Linear Regression is:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \ldots + \beta_n X_n$$

Where:
- Y= Dependent variable
- β_0= Intercept
- $\beta_1, \beta_2, \ldots, \beta_n$= Regression coefficients
- $X_1, X_2, \ldots, X_n$= Independent variables

Each coefficient represents the change in Y when that particular X increases by one unit while keeping other variables constant.

3. Working Principle

The model estimates coefficients by minimizing the error using the Least Squares Method.

Cost Function (MSE)

$$MSE = \frac{1}{n}\sum(Y_i - \hat{Y}_i)^2$$

The goal is to minimize this error to obtain the best-fit hyperplane.

4. Geometric Interpretation

- In Simple Linear Regression, the model fits a straight line.
- In Multiple Linear Regression, the model fits a plane (for 2 predictors) or a hyperplane (for more predictors).

5. Diagram Representation

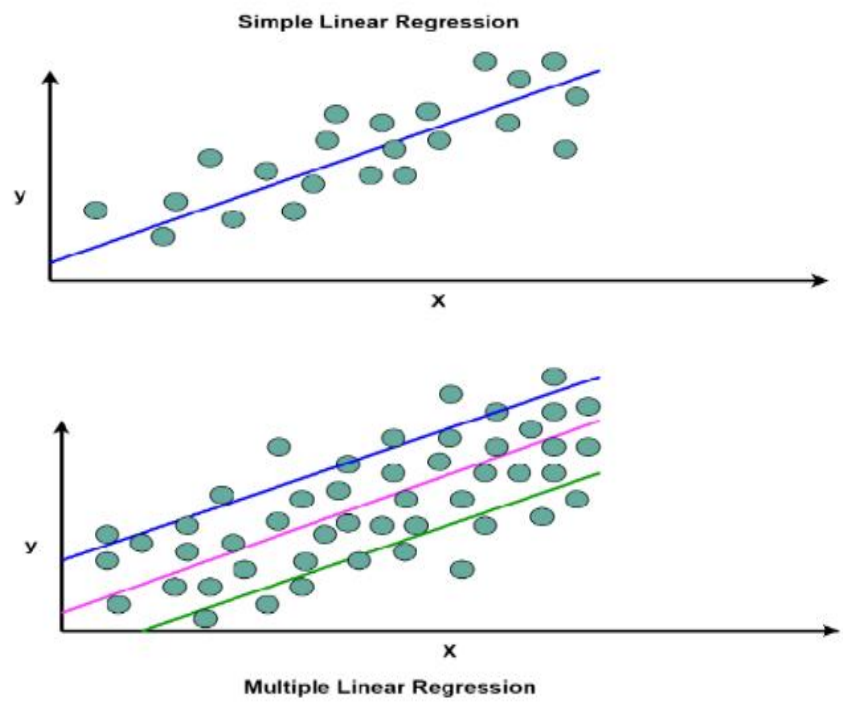

Figure 2: Multiple Linear Regression

6. Assumptions of Multiple Linear Regression

1. Linearity
2. Independence of errors
3. Homoscedasticity (constant variance of errors)
4. Normal distribution of residuals

5. No Multicollinearity (independent variables should not be highly correlated)

7. Model Evaluation Metrics
a. Mean Absolute Error (MAE)
Measures average absolute prediction error.

b. Mean Squared Error (MSE)
Penalizes large errors.

c. Root Mean Squared Error (RMSE)
Square root of MSE. Interpretable in original units.

d.R-Squared (R^2)
Indicates proportion of variance explained by the model.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

5. Adjusted R-Squared
Used in Multiple Regression to adjust for number of predictors:

$$Adjusted\ R^2 = 1 - \left(\frac{1 - R^2}{n - k - 1} \times (n - 1)\right)$$

Where:
- n = number of observations
- k = number of predictor

8. Algorithm Steps
1. Start
2. Import required libraries
3. Load dataset
4. Separate independent and dependent variables
5. Split data into training and testing sets
6. Fit Multiple Linear Regression model
7. Predict values
8. Evaluate using MAE, MSE, RMSE, R^2
9. Interpret coefficients
10. Stop

Oral Questions:
1. What is Multiple Linear Regression?
2. How is it different from Simple Linear Regression?
3. What is the role of regression coefficients?
4. What is multicollinearity?
5. Why is Adjusted R^2 used instead of R^2?

6. What are the assumptions of Multiple Linear Regression?
7. What happens if independent variables are highly correlated?
8. What is the difference between MAE and RMSE?
9. How do you interpret β_1 in the regression equation?
10. What is a hyperplane in regression?

Conclusion:

Assignment No. 3

Aim: Study and Implement Decision Tree.

Software/libraries required:

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn

Outcomes of Assignment:

1. Understand the concept and working of Decision Tree algorithm.
2. Differentiate between classification and regression trees.
3. Build and train a Decision Tree model using real datasets.
4. Interpret tree structure (root, nodes, leaves, splits).
5. Evaluate model performance using appropriate metrics.

Theory:

1. Introduction to Decision Tree

A **Decision Tree** is a supervised machine learning algorithm used for:

- **Classification** (e.g., Yes/No, Disease/No Disease)
- **Regression** (predicting continuous values)

It works by splitting the dataset into smaller subsets based on feature values, forming a tree-like structure.

2. Structure of Decision Tree

A Decision Tree consists of:

- **Root Node** → Starting point of the tree
- **Internal Nodes** → Decision-making nodes
- **Branches** → Outcomes of decisions
- **Leaf Nodes** → Final prediction/output

3. Working Principle

The algorithm selects the best feature for splitting the data using measures like:

a. Entropy

Entropy measures impurity in the dataset.

$$Entropy(S) = -\sum p_i \log_2(p_i)$$

- Value ranges from 0 to 1
- 0 → Pure node

- 1 → Maximum impurity

b. Information Gain

Information Gain measures reduction in entropy after splitting.

$$IG = Entropy(Parent) - Weighted\ Entropy(Children)$$

Feature with highest Information Gain is selected for splitting.

c. Gini Index

Used in CART algorithm.

$$Gini = 1 - \sum p_i^2$$

- Lower Gini → Better split

4. Types of Decision Trees

a. **Classification Tree** → Output is categorical
b. **Regression Tree** → Output is continuous

5. Diagram Representation

Example: Loan Approval Decision Tree

```
            [Income > 50K?]
               /      \
             Yes        No
            /            \
     [Credit Score > 700?]    Reject
        /        \
      Yes         No
     /             \
   Approve         Reject
```

Figure 3: Decision Tree

Explanation:

- Root node checks income.
- Next split based on credit score.
- Leaf nodes give final decision.

6. Algorithm Steps

1. Select the best feature using Information Gain or Gini Index
2. Split the dataset into subsets
3. Repeat splitting for each subset
4. Stop when:
    - Node becomes pure
    - Maximum depth reached
    - Minimum samples reached

Oral Questions:

1. What is a Decision Tree?
2. What is the difference between classification and regression trees?
3. What is entropy?
4. Define Information Gain.
5. What is Gini Index?
6. What is overfitting in Decision Tree?
7. What is pruning?
8. What are leaf nodes?
9. Why does Decision Tree not require feature scaling?
10. What are the advantages of Decision Trees over other algorithms?

Conclusion:

_____

_____

_____

_____

Assignment No. 4

Aim: Implement Logistic Regression using any dataset.

Software/libraries required:

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn

Outcomes of Assignment:

1. Understand the concept of Logistic Regression.
2. Differentiate between Linear and Logistic Regression.
3. Implement Logistic Regression on a real dataset.
4. Interpret sigmoid function and decision boundary.
5. Evaluate classification performance using standard metrics.

Theory:

1. Introduction to Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike Linear Regression (which predicts continuous values), Logistic Regression predicts probability of a class.

Example Applications:

- Disease Prediction (Yes/No)
- Email Spam Detection
- Loan Approval

2. Why Not Use Linear Regression for Classification?

Linear regression outputs continuous values that may go beyond 0 and 1.

Logistic Regression solves this problem by using the Sigmoid Function, which converts output into probability between 0 and 1.

3. Logistic Regression Model

Step 1: Linear Equation

$$Z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n$$

Step 2: Apply Sigmoid Function

$$\sigma(Z) = \frac{1}{1 + e^{-Z}}$$

This converts output into probability.

- If probability ≥ 0.5 → Class 1
- If probability < 0.5 → Class 0

5. Sigmoid Function Graph

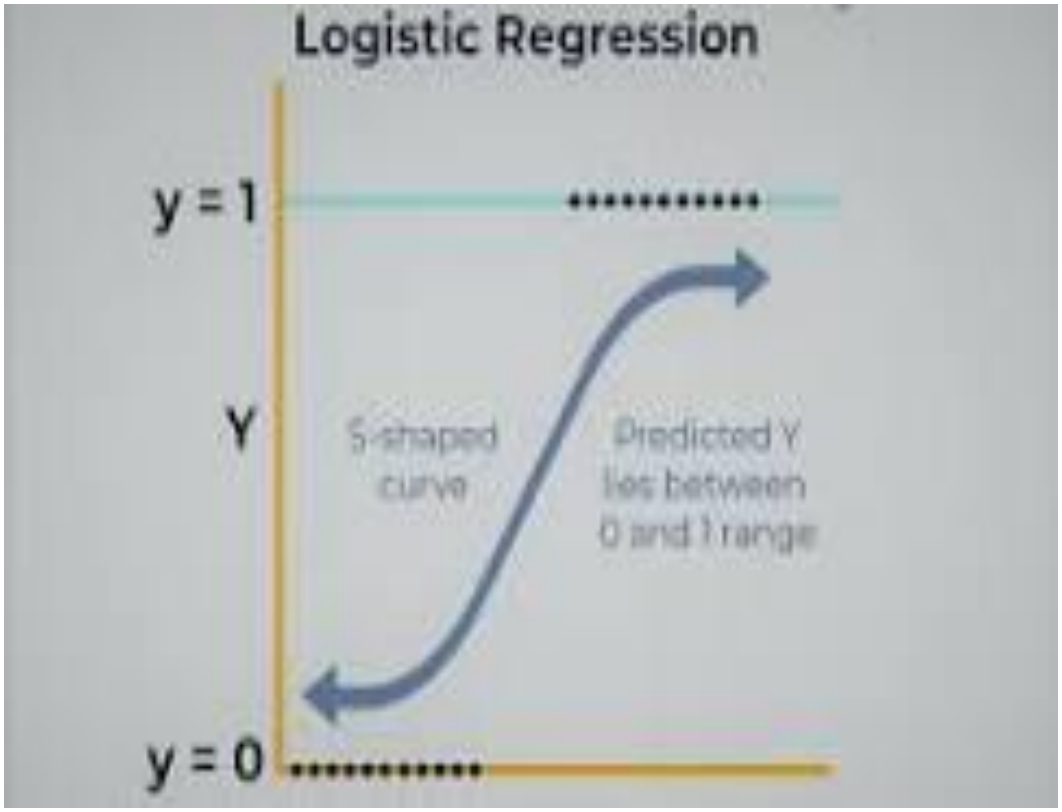

Figure 4: Sigmoid Function Graph

- S-shaped curve
- Output between 0 and 1

5. Decision Boundary

The decision boundary is the line (or hyperplane) that separates classes.
For one feature:

$$\beta_0 + \beta_1 X = 0$$

For two features:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$$

6. Cost Function (Log Loss)

Logistic Regression uses **Log Loss** instead of MSE.

$$Cost = -\frac{1}{n}\sum[y\log(\hat{y}) + (1-y)\log(1-\hat{y})]$$

This penalizes wrong predictions heavily.

7. Assumptions of Logistic Regression

1. Dependent variable is binary
2. Independent variables are independent of each other
3. Large sample size preferred
4. Little or no multicollinearity

8. Algorithm Steps

1. Import required libraries
2. Load dataset
3. Preprocess data (handle missing values, encoding)
4. Split into training and testing sets
5. Train Logistic Regression model
6. Predict probabilities
7. Convert probabilities into class labels
8. Evaluate model performance

Oral Questions:

1. What is Logistic Regression?
2. Why is it called regression if it is used for classification?
3. What is the sigmoid function?
4. What is decision boundary?
5. What is log loss?
6. What is the difference between Linear and Logistic Regression?
7. What is a confusion matrix?
8. What is precision and recall?
9. What happens if classes are imbalanced?
10. What is regularization in Logistic Regression?

Conclusion:

_____

_____

_____

_____

Assignment No. 5

Aim: Implement Naïve Bayes Classifier on data set of your choice. Test and Compare for Accuracy and Precision.

Software/libraries required:

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn

Outcomes of Assignment:

1. Understand the concept of Probabilistic Classification.
2. Explain Bayes' Theorem and conditional probability.
3. Implement Naïve Bayes classifier on a dataset.
4. Evaluate performance using Accuracy and Precision.
5. Compare classifier performance using evaluation metrics.

Theory:

1. Introduction to Naïve Bayes Classifier

Naïve Bayes is a supervised classification algorithm based on Bayes' Theorem with an assumption of independence among predictors.

It is widely used for:

- Spam Detection
- Sentiment Analysis
- Disease Prediction
- Document Classification

The term *"Naïve"* comes from the assumption that all features are independent of each other.

2. Bayes' Theorem

Bayes' Theorem is given by:

$$P(C \mid X) = \frac{P(X \mid C) \cdot P(C)}{P(X)}$$

Where:

- $P(C \mid X) \rightarrow$ Posterior Probability
- $P(X \mid C) \rightarrow$ Likelihood
- $P(C) \rightarrow$ Prior Probability
- $P(X) \rightarrow$ Evidence

The classifier predicts the class with maximum posterior probability.

3. Types of Naïve Bayes

1. **Gaussian Naïve Bayes** → For continuous data
2. **Multinomial Naïve Bayes** → For text classification
3. **Bernoulli Naïve Bayes** → For binary features

4. Working Principle

Step 1: Calculate Prior Probability for each class
Step 2: Calculate Likelihood for each feature
Step 3: Apply Bayes' Theorem
Step 4: Choose class with highest posterior probability

5. Diagram Representation

Naïve Bayes Classification Process

```
          Input Features
        (X1, X2, X3, X4)
              |
              |
        Apply Bayes Theorem
              |
    --------------------------------
    |                              |
P(Class A | X)                P(Class B | X)
    |                              |
    --------------------------------
              |
          Select Maximum
              |
          Predicted Class
```

Figure 5: Naïve Bayes Classification Process

6. Assumptions of Naïve Bayes

1. Features are independent
2. All predictors contribute equally
3. Large dataset improves performance

7. Model Evaluation

To test and compare performance, we use:

a. Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Measures overall correctness of model.

b.Precision

$$Precision = \frac{TP}{TP + FP}$$

Measures how many predicted positives are actually correct.

8. Steps to Implement

1. Import required libraries
2. Load dataset (e.g., Iris dataset)
3. Split dataset into training and testing sets
4. Train Gaussian Naïve Bayes model
5. Predict test data
6. Generate confusion matrix
7. Calculate Accuracy

Oral Questions:

1. What is Naïve Bayes classifier?
2. Why is it called "Naïve"?
3. State Bayes' Theorem.
4. What are Prior and Posterior probabilities?
5. What is the independence assumption?
6. What is the difference between Gaussian and Multinomial Naïve Bayes?
7. What is a confusion matrix?
8. Define Accuracy and Precision.
9. When is Precision more important than Accuracy?
10. What are the limitations of Naïve Bayes?

Conclusion:

Assignment No. 6

Aim: Implement K Nearest Neighbour (KNN) classification.

Software/libraries required:

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn

Outcomes of Assignment:

1. Understand the working principle of K-Nearest Neighbour (KNN) algorithm.
2. Explain distance-based classification methods.
3. Implement KNN classification using a dataset.
4. Select optimal value of K.
5. Evaluate model performance using classification metrics.

Theory:

1. Introduction to KNN

K-Nearest Neighbour (KNN) is a supervised machine learning algorithm used for:

- Classification
- Regression

It is a lazy learning algorithm, meaning it does not build a model during training. Instead, it stores the dataset and makes predictions during testing.

2. Working Principle of KNN

KNN works based on similarity (distance measure).

Steps:

1. Choose the value of K (number of neighbors).
2. Calculate distance between test point and all training data points.
3. Select K nearest neighbors.
4. Assign the class based on majority voting.

3. Distance Metrics Used

a. Euclidean Distance

$$d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

Most commonly used distance metric.

b.Manhattan Distance

$$d = | x_1 - y_1 | + | x_2 - y_2 |$$

c.Minkowski Distance
Generalized form of distance metric.

4. Diagram Representation
KNN Classification (K = 3)

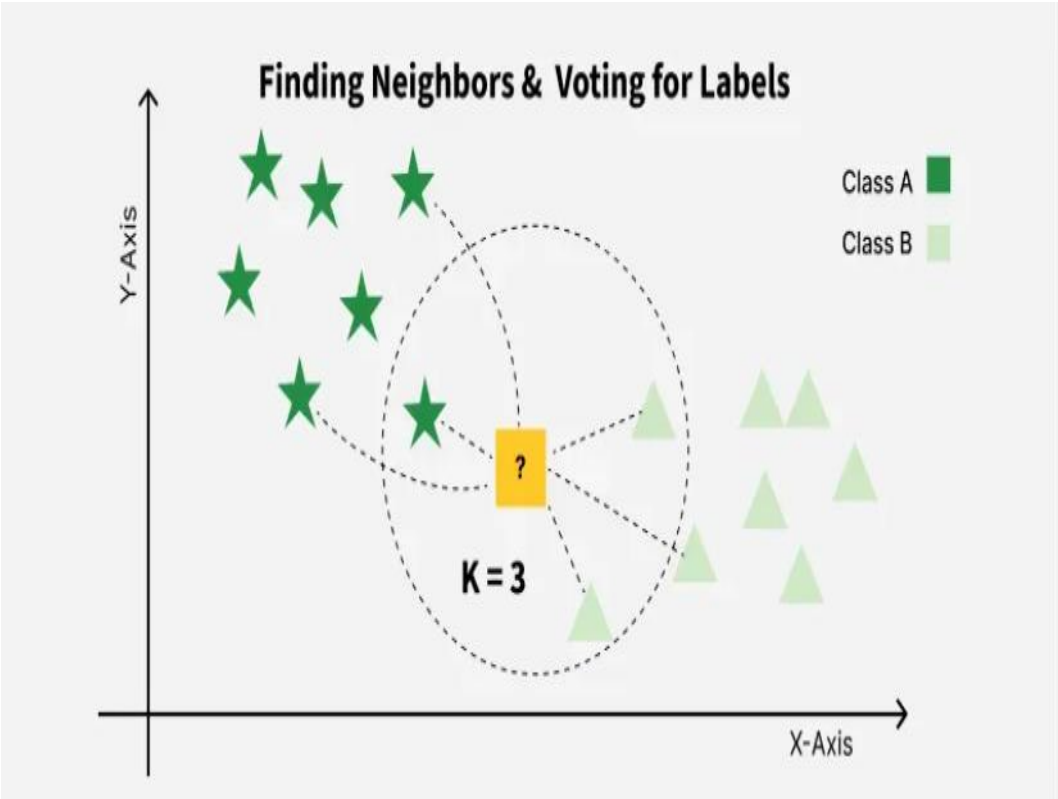

Figure 6: K Nearest Neighbour Classification

5. Choosing Optimal K

- Small K → High variance, may overfit
- Large K → High bias, may underfit

Typically:

- Use odd value (to avoid tie)
- Use cross-validation to select best K

6. Algorithm Steps

1. Import required libraries
2. Load dataset
3. Preprocess data (feature scaling important)

4. Split into training and testing sets
5. Choose K value
6. Train KNN model
7. Predict test data
8. Evaluate performance

7. Importance of Feature Scaling

Since KNN is distance-based, features must be scaled using:

- Standardization
- Normalization

Without scaling, features with larger values dominate.

Oral Questions:

1. What is KNN algorithm?
2. Why is KNN called a lazy learner?
3. What is the role of K in KNN?
4. What is Euclidean distance?
5. Why is feature scaling important in KNN?
6. What happens if K is too small?
7. What happens if K is too large?
8. What are the advantages of KNN?
9. How do you choose the optimal K value?
10. What is the difference between KNN classification and regression?

Conclusion:

Assignment No. 7

Aim: Implement Support Vector Machine algorithm.

Software/libraries required:

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn

Outcomes of Assignment:

1. Understand the concept of Support Vector Machine (SVM).
2. Explain the idea of margin and support vectors.
3. Differentiate between linear and non-linear SVM.
4. Implement SVM using a dataset.
5. Evaluate model performance using classification metrics.

Theory:

1. Introduction to Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised machine learning algorithm used for:

- Classification
- Regression

SVM aims to find the optimal hyperplane that separates data points of different classes with maximum margin.

2. Hyperplane Concept

For:

- 2D data → Hyperplane is a line
- 3D data → Hyperplane is a plane
- Higher dimensions → Hyperplane

The goal is to find the best separating boundary.

3. Maximum Margin Principle

SVM chooses a hyperplane such that:

- Distance between hyperplane and nearest data points is maximum.
- These nearest points are called Support Vectors.

4. Mathematical Representation

Hyperplane equation:

$$w^T x + b = 0$$

Where:

- w= weight vector
- x= feature vector
- b= bias

Margin:

$$\text{Margin} = \frac{2}{\| w \|}$$

Maximizing margin improves generalization.

5. Diagram Representation (Linear SVM)

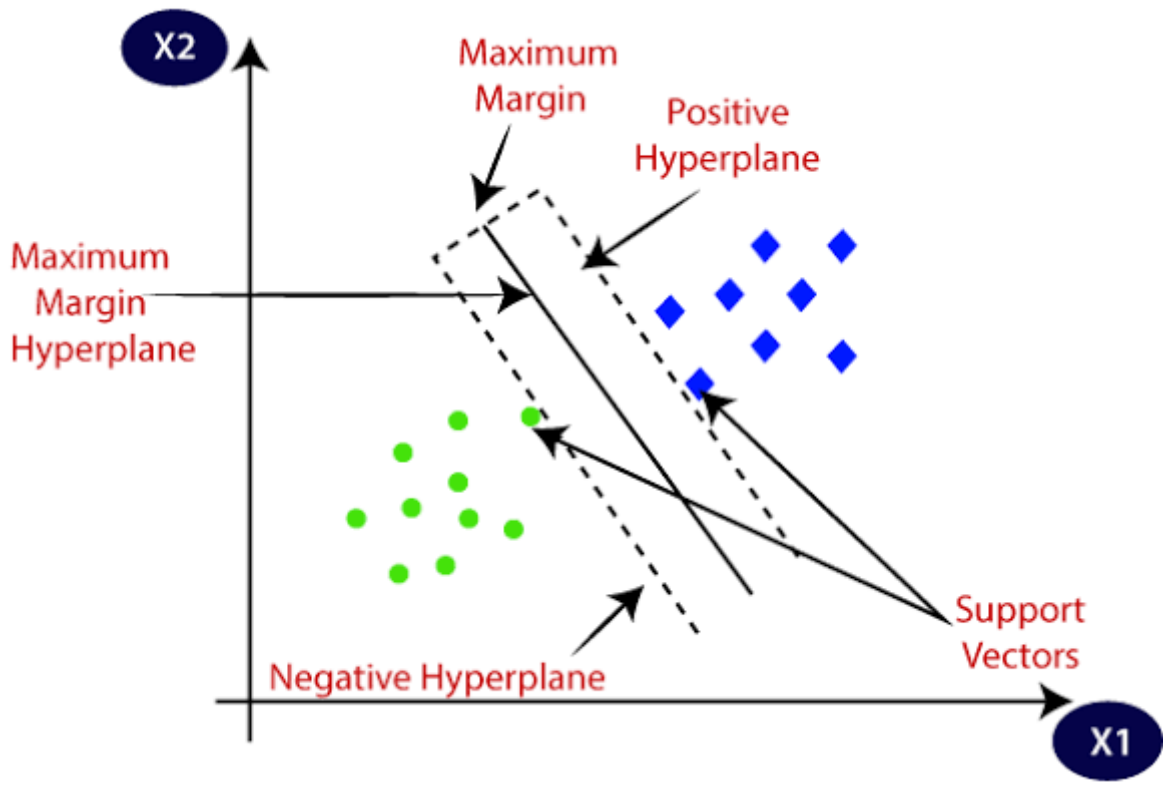

Figure 7: Support Vector Machine

6. Hard Margin vs Soft Margin

Hard Margin

- Perfect separation
- No misclassification allowed
- Used when data is linearly separable

Soft Margin

- Allows some misclassification

- Introduces regularization parameter **C**
- Used for real-world noisy data

7. Kernel Trick (Non-Linear SVM)
When data is not linearly separable, SVM uses Kernel functions to transform data into higher dimensions.
Common Kernels:
- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF) Kernel
- Sigmoid Kernel

RBF Kernel formula:

$$K(x, x') = e^{-\gamma ||x-x'||^2}$$

8. Algorithm Steps
1. Import required libraries
2. Load dataset
3. Preprocess data (feature scaling required)
4. Split into training and testing sets
5. Choose kernel (linear or RBF)
6. Train SVM model
7. Predict test data
8. Evaluate performance

9. Importance of Feature Scaling
SVM is sensitive to feature magnitude.
Therefore, scaling (Standardization) is essential.

Oral Questions:
1. What is Support Vector Machine?
2. What is a hyperplane?
3. What are support vectors?
4. What is margin in SVM?
5. What is the difference between hard margin and soft margin?
6. What is the role of parameter C?
7. What is kernel trick?
8. Why is feature scaling important in SVM?
9. What is RBF kernel?
10. What are the advantages of SVM?

Conclusion:

_____

_____

_____

_____

Assignment No. 8

Title: Classification Prediction Using K-Means Clustering (K = 3) for Two-Variable Dataset

Aim: Given the following data, which specify classifications for nine combinations of VAR1and VAR2 predict a classification for a case where VAR=0.906 and VAR2=0.606, using the result of k-means clustering with 3 means (i.e., 3centroids)

VAR1	VAR2	CLASS
1.713	1.586	0
0.180	1.786	1
0.353	1.240	1
0.940	1.566	0
1.486	0.759	1
1.266	1.106	0
1.540	0.419	1
0.459	1.799	1
0.773	0.186	1

Software/libraries required:

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Outcomes of Assignment:

1. Understand the concept of K-Means Clustering.
2. Apply K-Means with K = 3 centroids on a dataset.
3. Compute Euclidean distance for cluster assignment.
4. Identify cluster centroids and classify new data points.
5. Interpret clustering results for prediction.

Theory:

1. Introduction to K-Means Clustering

K-Means is an unsupervised machine learning algorithm used to group similar data points into clusters.

In this problem:

- Input Features → VAR1, VAR2
- Number of Clusters → K = 3
- Goal → Assign new data point (0.906, 0.606) to one of the clusters

Although class labels are provided, clustering is performed based only on feature similarity.

2. Working Principle of K-Means

Step 1: Choose K = 3

Select 3 initial centroids.

Step 2: Assign Data Points

Assign each point to nearest centroid using Euclidean Distance:

$$d = \sqrt{(x_1 - c_1)^2 + (x_2 - c_2)^2}$$

Step 3: Update Centroids

Compute new centroid by taking mean of all points in cluster:

$$C = \left(\frac{\sum x}{n}, \frac{\sum y}{n}\right)$$

Step 4: Repeat

Repeat steps 2 and 3 until centroids stop changing.

Dataset

VAR1	VAR2	CLASS
1.713	1.586	0
0.180	1.786	1
0.353	1.240	1
0.940	1.566	0
1.486	0.759	1
1.266	1.106	0
1.540	0.419	1
0.459	1.799	1
0.773	0.186	1

New point to classify:

$$(VAR1, VAR2) = (0.906, 0.606)$$

4. Diagram Representation
K-Means Clustering (K = 3)

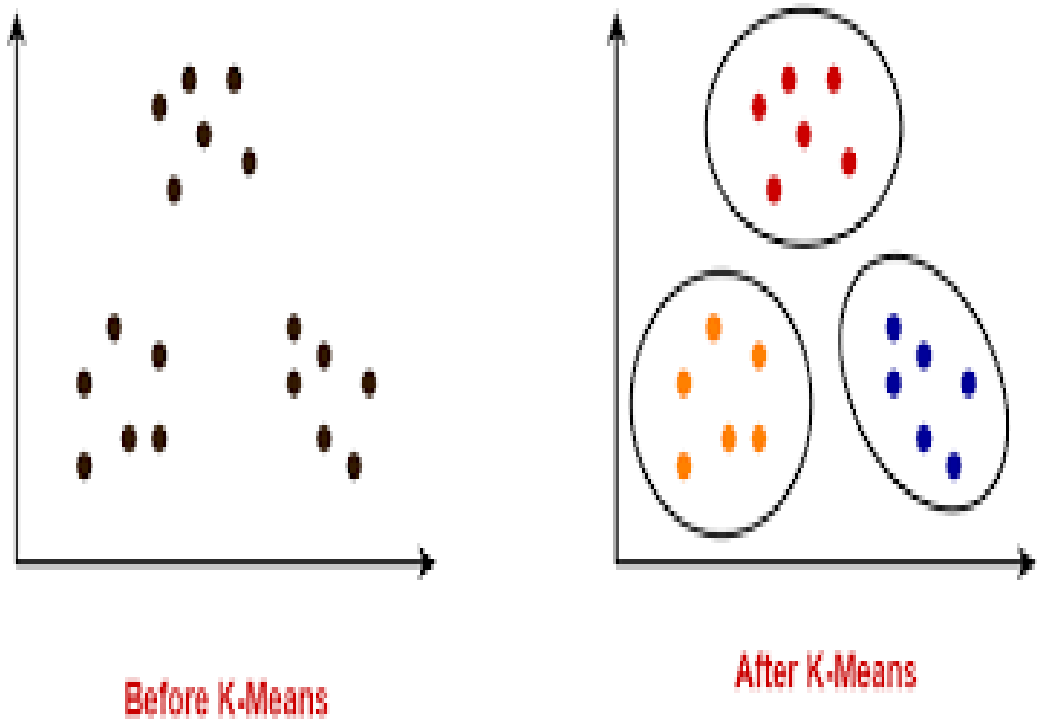

Figure 8: K Means Clustering

5. Classification of New Data Point
After running K-Means with K = 3:

1. Compute centroids of 3 clusters.
2. Calculate distance of (0.906, 0.606) from each centroid.
3. Assign it to nearest cluster.
4. Identify majority class of that cluster.

Final Prediction:
Based on distance comparison,
The new point (0.906, 0.606) belongs to the cluster dominated by **Class 1**.
Hence, **Predicted Class = 1**

Oral Questions:

1. What is K-Means clustering?
2. Is K-Means supervised or unsupervised?
3. What is a centroid?
4. What distance measure is commonly used in K-Means?
5. How do you update centroids?
6. What happens if K is chosen incorrectly?

7. Why is K-Means sensitive to initialization?
8. How do you classify a new data point using clusters?
9. What is the stopping condition of K-Means?
10. What are limitations of K-Means algorithm?

Conclusion:

Assignment No. 9

Aim: Unsupervised Learning: Implement K-Means Clustering and Hierarchical clustering on proper data set of your choice. Compare their Convergence.

Software/libraries required:

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn
- SciPy

Outcomes of Assignment:

1. Understand the concept of Unsupervised Learning.
2. Implement K-Means and Hierarchical Clustering algorithms.
3. Visualize clusters using scatter plots and dendrograms.
4. Compare convergence behavior of both algorithms.
5. Interpret clustering results and evaluate cluster quality.

Theory:

1. Introduction to Unsupervised Learning

Unsupervised learning is used when data does not contain labeled outputs.
The algorithm identifies patterns and groups similar data points.
Clustering is a major unsupervised learning technique.
Two important clustering methods:

- **K-Means Clustering**
- **Hierarchical Clustering**

2. K-Means Clustering

Working Principle

K-Means partitions data into K clusters based on distance from centroids.

Steps:

1. Select number of clusters (K).
2. Initialize K centroids randomly.
3. Assign each data point to nearest centroid.
4. Recalculate centroids.
5. Repeat until centroids stop changing (convergence).

Objective Function
K-Means minimizes:

$$WCSS = \sum || x_i - c_j ||^2$$

(WCSS = Within Cluster Sum of Squares)

Diagram: K-Means

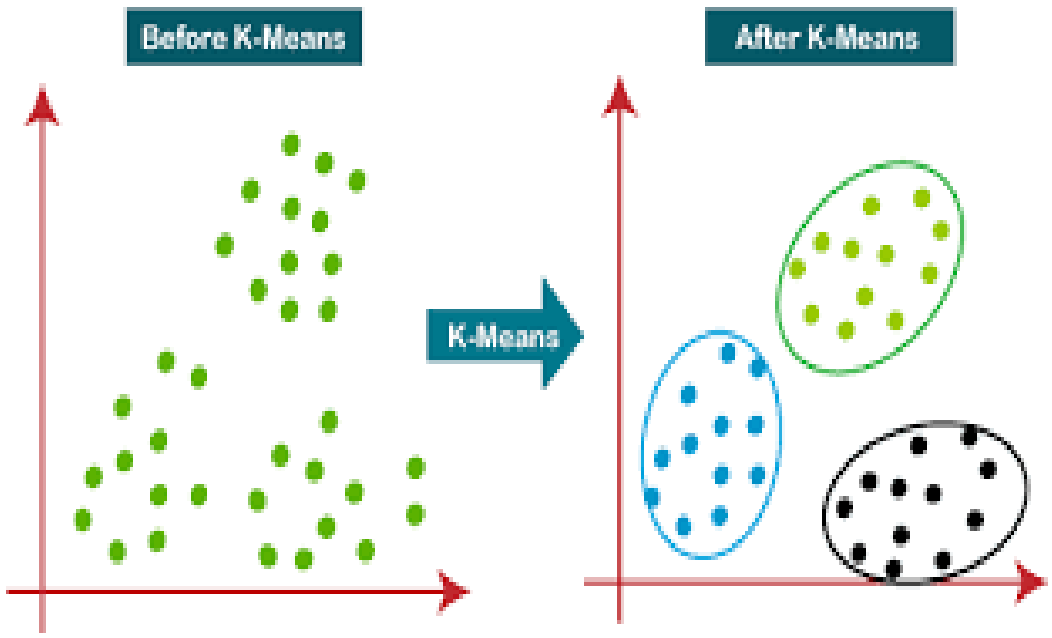

Figure 9.1: K Means Clustering

Convergence in K-Means
- Iterative algorithm
- Converges when centroid movement becomes zero
- May converge to local minimum
- Faster compared to hierarchical clustering

3. Hierarchical Clustering
Hierarchical clustering builds a tree-like structure (dendrogram).
Two types:
1. Agglomerative (Bottom-Up)
2. Divisive (Top-Down)

Most commonly used: Agglomerative Clustering

Agglomerative Clustering Steps
1. Treat each data point as a separate cluster.

2. Compute distance between clusters.
3. Merge two closest clusters.
4. Repeat until one cluster remains.

Distance between clusters can be:

- Single Linkage
- Complete Linkage
- Average Linkage

Diagram: Dendrogram

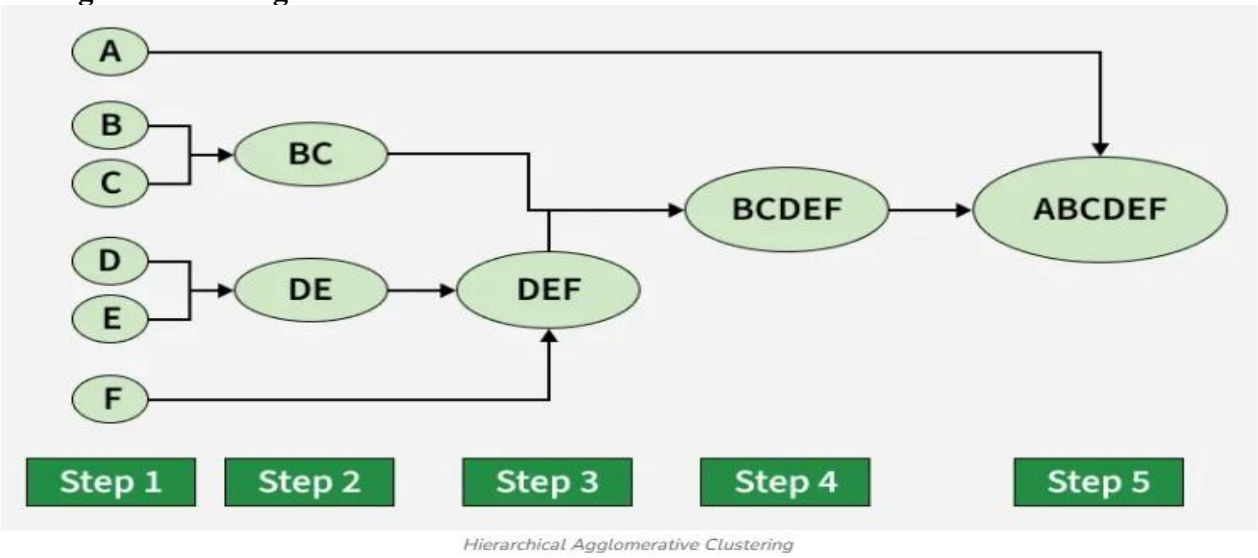

Figure 9.2: Hierarchical Agglomerative Clustering

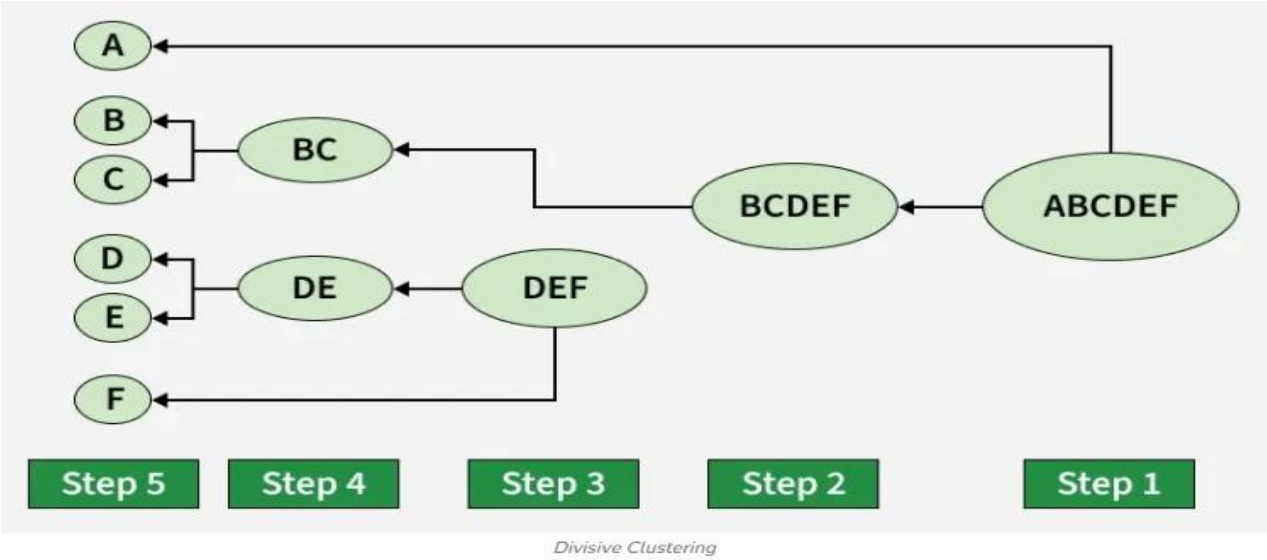

Figure 9.3: Divisive Clustering

Convergence in Hierarchical Clustering

- No iterative centroid updates
- Deterministic (no random initialization)
- Does not "converge" in traditional sense
- Forms full cluster tree
- Computationally expensive for large datasets

Comparison of Convergence

Feature	K-Means	Hierarchical
Nature	Iterative	Tree-based
Convergence	Stops when centroids stabilize	Builds full dendrogram
Speed	Fast	Slower
Scalability	Good for large datasets	Not suitable for very large datasets
Initialization	Random centroids	No random initialization
Risk	Local minima	Deterministic result

Oral Questions:

1. What is Unsupervised Learning?
2. What is clustering?
3. Explain K-Means algorithm.
4. What is WCSS?
5. What is convergence in K-Means?
6. What is Hierarchical clustering?
7. What is a dendrogram?
8. Difference between Agglomerative and Divisive clustering?
9. Which algorithm is faster and why?
10. How do you choose optimal number of clusters?

Conclusion:

_____

_____

_____

_____

Assignment No. 10

Aim: Study and Implement Bagging/Boosting using Random Forests.

Software/libraries required:

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn

Outcomes of Assignment:

1. Understand Ensemble Learning concepts.
2. Explain Bagging and Boosting techniques.
3. Implement Random Forest (Bagging-based ensemble).
4. Compare ensemble methods with single Decision Tree.
5. Evaluate model performance using classification metrics.

Theory:

1. Introduction to Ensemble Learning

Ensemble Learning combines multiple weak learners to create a strong predictive model.
Basic idea:
"Multiple models together perform better than a single model."
Two major ensemble techniques:

- Bagging
- Boosting

2. Bagging (Bootstrap Aggregating)

Bagging reduces **variance** by training multiple models on different random samples of data.
Steps:

1. Create multiple bootstrap samples from original dataset.
2. Train a model (e.g., Decision Tree) on each sample.
3. Combine predictions using:
    - Majority voting (classification)
    - Averaging (regression)

3. Random Forest (Bagging-Based Method)

Random Forest is an ensemble of Decision Trees built using:

- Bootstrap sampling
- Random feature selection

Each tree:
- Uses random subset of data
- Uses random subset of features

Final output:
- Majority vote (classification)
- Average prediction (regression)

Diagram: Random Forest (Bagging)

```
      Original Dataset
            |
  ---------------------------------
  |         |          |
 Sample 1   Sample 2    Sample 3
  |         |          |
  Tree 1    Tree 2     Tree 3
  |         |          |
  ---------------------------------
            Voting
            |
        Final Prediction
```

Figure 10.1: Random Forest(Bagging)

4. Boosting

Boosting reduces **bias** by training models sequentially.
Each new model focuses more on previously misclassified points.
Popular boosting algorithms:
- AdaBoost
- Gradient Boosting
- XGBoost

Boosting Working Principle

1. Train first weak learner.
2. Increase weight of misclassified points.
3. Train next learner on weighted data.
4. Combine all learners using weighted voting.

```
Data → Weak Learner 1
        |
    Misclassified?
        ↓
Weak Learner 2 (focuses more)
        |
Weak Learner 3
        |
    Weighted Voting
        |
   Final Strong Model
```

Figure 10.2: Boosting

5. **Difference Between Bagging and Boosting**

Feature	Bagging	Boosting
Goal	Reduce variance	Reduce bias
Training	Parallel	Sequential
Overfitting	Less prone	Can overfit
Example	Random Forest	AdaBoost

6. Random Forest Algorithm Steps

1. Import libraries
2. Load dataset
3. Split into training and testing sets
4. Initialize Random Forest classifier
5. Train model
6. Predict output
7. Evaluate performance

Oral Questions:

1. What is Ensemble Learning?
2. What is Bagging?
3. What is Boosting?
4. How does Random Forest work?
5. What is bootstrap sampling?
6. Difference between Bagging and Boosting?

7. Why is Random Forest better than Decision Tree?
8. What is feature randomness in Random Forest?
9. What are advantages of Boosting?
10. What are limitations of ensemble methods?

Conclusion:

_____
_____
_____
_____

Content Beyond Syllabus

Aim: To classify simple binary data using a quantum circuit as a feature map in a hybrid quantum-classical approach.

Software/libraries required:

- Python
- Qiskit
- Qiskit Aer
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Outcomes of Assignment:

1. Understand the basics of quantum computing concepts (qubits, gates, superposition).
2. Explain hybrid quantum-classical machine learning models.
3. Design a quantum feature map circuit for binary classification.
4. Integrate quantum circuits with classical classifiers.
5. Evaluate classification performance using standard metrics.

Theory:

1. Introduction to Quantum Machine Learning (QML)

Quantum Machine Learning combines:

- Quantum computing (for feature transformation)
- Classical machine learning (for optimization and prediction)

In this experiment, we use a quantum circuit as a feature map, and then apply a classical classifier. This is called a Hybrid Quantum-Classical Model.

2. Basic Quantum Concepts

Qubit

Unlike classical bit (0 or 1), a qubit exists in superposition:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$$

Where:

- α, β are probability amplitudes
- $|\alpha|^2 + |\beta|^2 = 1$

Quantum Gates

Common gates:

- Hadamard (H) → Creates superposition
- Pauli-X → Bit flip
- Rotation gates (RX, RY, RZ)
- CNOT → Entanglement

3. Quantum Feature Map

A feature map encodes classical data into quantum states.

For example:

- Input feature x is encoded using rotation gate:

$$RY(x)$$

This maps classical feature to quantum state.

Feature Encoding Circuit

```
|0> —— H —— RY(x1) ——●————————— Measure
                   |
|0> —— H —— RY(x2) ——⊕————————— Measure
```

1. Apply Hadamard gate → superposition
2. Apply rotation gate RY(x) to encode feature
3. Apply CNOT → entanglement
4. Measure output

This transforms input data into higher-dimensional quantum feature space.

4. Hybrid Quantum-Classical Approach

Step 1: Classical Data

Binary dataset (e.g., two features and class label).

Step 2: Quantum Feature Map

Encode input features into quantum circuit.

Step 3: Quantum Measurement

Obtain expectation values.

Step 4: Classical Classifier

Use classical algorithm (e.g., SVM or Logistic Regression).

```
Classical Data
     |
Quantum Feature Map
     |
Quantum Measurement
     |
Classical Optimizer
     |
 Final Prediction
```

6. Classification Process

1. Prepare dataset
2. Normalize input features
3. Encode data into quantum circuit
4. Simulate circuit using Qiskit Aer
5. Extract measurement results
6. Train classical classifier
7. Evaluate performance

Oral Questions:

1. What is a qubit?
2. What is superposition?
3. What is a quantum feature map?
4. What is entanglement?
5. What is hybrid quantum-classical learning?
6. Why do we use rotation gates for encoding?
7. What is the role of measurement in quantum circuits?
8. What are advantages of quantum feature space?
9. What are limitations of current quantum hardware?
10. How does quantum classification differ from classical classification?

Conclusion:

_____

_____

_____

_____